Mahesha's Brief Professional Bio

A Compiler Engineer's Journey

Mahesha S

mahesha.shivamallappa@gmail.com

October 2026

I'm Mahesha — a compiler engineer with over 20 years of experience building production compilers across the stack, spanning Clang, MLIR, and LLVM, for CPU, GPU, and AI accelerator architectures. What drives me is simple: tackling hard problems, figuring out how things really work, and building solutions from scratch — and I find just as much satisfaction in helping the engineers around me grow and take on greater responsibility. Across these two decades I've had my share of ups and downs, and if there's one thing that has carried me through the uncertain stretches, it's a positive frame of mind and the courage to keep moving forward regardless.

To start with, I hold a Master's in Computer Science and a Bachelor's in Mathematics, both from the University of Mysore, and I graduated in 2003.

After graduating, I wanted to get into research, and with that in mind I took up a teaching position at an engineering college in Mysore, teaching discrete mathematics and C programming, for about a year before starting my industry journey.

But family circumstances meant I needed to step into industry, so I set research aside and decided to work in the compiler domain instead. Compilers sit at a rare intersection of theory and hands-on engineering — the rigorous theory behind parsing, type systems, and optimization on one side, and the practical discipline of making all of it run correctly and efficiently on real hardware on the other. Nothing felt as satisfying as watching a program survive that entire journey from source code down to the exact machine instructions actually executed on the hardware — and that fascination is what led me to start looking specifically for compiler engineer roles.

My first industry job was at a small startup, DACS Software, working on a tool that translated assembly code between two DSP architectures for a collaboration with CEVA. The two architectures executed instructions in fundamentally different ways, so a naive, instruction-by-instruction translation would have produced slow code. I designed a strategy to regroup the translated instructions in a way the target architecture could execute efficiently, and spent the rest of that year chasing successive optimizations to close the performance gap, at one point hand-porting a few performance-critical applications myself under a tight deadline.

From DACS, I moved to HP, stepping into industry-strength production compiler work on their in-house C++ front-end for the PA-RISC architecture. C++ was evolving quickly at the time, and the aging Lex/Yacc-based front-end was straining to keep up — I spent my early time there chasing parser and template-semantics issues in a codebase that hadn't been built for the language C++ was becoming. Eventually HP decided to replace that front-end with the industry-standard EDG front-end, and I worked with a small group of senior engineers on the hard part of that migration: building a bridge that lowered EDG's target-neutral ASTs into HP's native, PA-RISC-ABI-correct representation without breaking binary compatibility.

From HP, I joined Siemens' research lab, where I wore two very different hats. On the product side, I took ownership of building a new compiler front-end for Structured Text, the language used to program Siemens PLCs — my first time leading a team with no prior compiler background, and a good early lesson in what it takes to mentor and guide people through genuinely unfamiliar territory, the kind of hands-on leadership that got them to deliver the product within the business timeline. On the research side, I worked on an exploratory project asking whether a workload's compute characteristics could be analyzed automatically and retargeted, via a source-to-source compiler, onto the right programming model — CUDA, OpenMP, or otherwise — for emerging heterogeneous hardware. I also spent time evaluating Intel's early Many Integrated Core architecture for Siemens' own workloads. Not all research survives changing business priorities, but both projects shaped how I think about compilers for non-traditional hardware, which turned out to matter a great deal later in my career.

After a four-year stint at Siemens, I moved to AMD in mid-2012, at a time when the open-source Clang/LLVM ecosystem was clearly emerging as the future of compiler infrastructure. I got the chance to work on it there, as part of AMD's migration from their older Open64 compiler infrastructure to Clang/LLVM. I proposed and began implementing OpenMP support in Clang's front-end — including presenting the design to the LLVM community for review — but changing priorities at AMD cut that effort short before it shipped, and the work was eventually picked up and carried forward by engineers at Intel.

Left with little choice after that sudden, unexpected turn, I landed at Samsung, where I worked on building a tool that automatically generated random test cases to stress-test their OpenCL compiler for Samsung's mobile GPUs. I built a floating-point expression testing methodology that validated numerical results within a configurable tolerance, and designed a data-flow-analysis-based pointer-range algorithm that let the random test generator produce pointer expressions that were safe and semantically valid rather than accidentally undefined — enough of a result that I wrote up an internal paper on it at the time, an idea I later revisited and published as one of the technical papers listed on this site.

After two years there, family responsibilities needed more of my attention, and the scarcity of strong compiler engineering roles in India at the time left me genuinely uncertain about where my career was headed. I decided to step away from work for over a year, take stock, and start with a clean slate.

Reflecting on it, I decided to pivot toward distributed systems and ML, so I joined Jivox, where I designed and integrated a recommender system into their ad-personalization platform and later built a sharding scheme to keep the system's databases scaling with its data. From there I moved to Qubole, where I evaluated and prototyped how a distributed deep learning framework could sit on top of Qubole's existing data-lake infrastructure.

It was while I was at Qubole that I noticed AI hardware companies multiplying almost overnight, and with them, a fresh wave of genuinely hard compiler problems. I rejoined AMD in early 2019 to work on their Clang/LLVM-based GPU compiler, and the next three and a half years turned out to be some of the most varied of my career. I helped stabilize HIP-Clang, the compiler at the heart of AMD's open-source ROCm platform, as AMD pushed it as a serious alternative to CUDA — including working with AMD's internal PyTorch team to make HIP-Clang the default compiler for PyTorch's AMD GPU backend. I also designed support for Cooperative Groups, a GPU programming model that lets programs coordinate groups of threads more flexibly, and built a compiler optimization that meaningfully cut how much on-chip memory each GPU program needed to reserve — a change that made it into the upstream LLVM project, though it was later disabled by default after it caused issues in some other workloads, a reminder of how far compiler changes can ripple. Alongside that work, I

tackled a steady stream of other hard, business-critical problems across AMD's GPU compiler stack — the kind of issues that don't make for a tidy story, but that products depend on getting right.

After that stretch at AMD, I joined Intel to work on the AI compiler for what was then Habana Gaudi hardware. One of the more interesting problems I solved there came from the hardware itself changing underneath the compiler: as AI models kept growing, Intel needed to widen a core piece of its accelerator's internal tensor representation, in a way that had no natural equivalent in the existing compiler infrastructure. I designed an approach that let the compiler keep working with its existing, well-understood data types on the surface, while quietly handling the new hardware requirement underneath — so nothing else built on top of the compiler needed to change. It shipped successfully and held up well as customer adoption grew.

A business restructuring then moved our team from AI-accelerator work to Intel's GPU compiler, and later onto an MLIR-based compiler whose job was to fuse AI framework computation graphs together and generate efficient GPU kernel code from them. I helped stabilize that compiler in its early days, then spent a good part of my remaining time at Intel building its support for the newer, lower-precision number formats — FP8, FP4, and the like — that have become central to running AI models efficiently, including the quantization and scaled matrix-multiply (MMA) work needed to actually use them well. That meant reconciling how several different layers of the stack, from AI frameworks down to the hardware itself, each represented these numbers slightly differently, in an area that kept evolving as the hardware did. After four years at Intel, that project came to an end when another round of business restructuring canceled it.

Looking back at my 20+ years of industry career — working on compilers targeted for DSPs, CPUs, GPUs, and now AI accelerators — what's carried me through it all was never fame, position, or power. But it's been the curiosity, the pull of genuine business needs, and the plain pleasure of taking a hard, often ambiguous problem, understanding it from first principles, and building something real out of it — almost always alongside colleagues I genuinely enjoyed working with and learned from. It's that same unwavering curiosity — paired with a stubborn, positive refusal to be defeated by uncertainty — that carried me through the job losses from restructuring, the unavoidable career breaks, and the sudden turns along the way. And it's that same curiosity that continues to guide me into whatever comes next.